

STL: Stack, Queue & Iterators

A short reference for interviews. C++11 baseline, C++17/20 marked where it matters.

What this covers

Part	Topic	Why it comes up
1	Iterators	Category questions, invalidation bugs, erase-in-loop, writing one by hand
2	<code>std::stack</code>	Adapter design, why <code>pop()</code> returns void, swapping the underlying container
3	<code>std::queue</code>	Container requirements, front/back lifetime, <code>priority_queue</code> comparators
4	Drill	Twenty questions with the answers an interviewer is listening for

Assumed: you can read a template declaration and you know what vector, deque and list cost. Everything else is spelled out.

Code compiles as written unless a line is marked with `// ERROR` or `// UB`.

A note on how to read the complexity column: $O(1)$ with no qualifier means genuinely constant. *Amortised* $O(1)$ means occasional reallocation, cheap on average. If an interviewer asks for worst case and you say amortised, say the word amortised out loud.

1. Iterators

An iterator is a generalised pointer. It is any object that answers `*it`, `++it` and `it == other`. That is the whole contract at the bottom of the hierarchy; everything above adds guarantees.

Algorithms in `<algorithm>` are written against categories, not containers. That is why `std::sort` refuses `std::list`: sort needs random access, a list only offers bidirectional. The list carries its own `.sort()` member for exactly that reason.

1.1 The categories

Category	Adds	Typical source
Input	read once, single pass, <code>*it</code> then <code>++it</code>	<code>istream_iterator</code>
Output	write once, single pass	<code>ostream_iterator</code> , <code>back_inserter</code>
Forward	multi-pass, can re-read the same position	<code>forward_list</code> , <code>unordered_*</code>
Bidirectional	<code>--it</code>	<code>list</code> , <code>set</code> , <code>map</code> , <code>multiset</code> , <code>multimap</code>
Random access	<code>it + n</code> , <code>it - n</code> , <code>it[n]</code> , <code>it < other</code> , distance in $O(1)$	<code>vector</code> , <code>deque</code> , <code>array</code> , <code>string</code>
Contiguous (C++20)	elements really are adjacent in memory, <code>&*(it+n) == &*it + n</code>	<code>vector</code> , <code>array</code> , <code>string</code> , <code>span</code>

Each category is a refinement of the one above it, so a random-access iterator satisfies every algorithm that only asks for input. Contiguous is the C++20 addition; before that `vector` and `deque` were both just "random access", which hid the fact that only one of them can be handed to a C API expecting `T*`.

Handy discriminator: `deque` is random access but **not** contiguous. It is the standard counter-example and interviewers like it.

1.2 Getting around

```
std::vector<int> v{10, 20, 30, 40, 50};

auto it = v.begin();
std::advance(it, 3);           // in-place, works on any category
auto nx = std::next(it);       // returns a copy, does not move `it`
auto pv = std::prev(it, 2);     // bidirectional or better
auto d = std::distance(v.begin(), it); // 3

// C++11 free functions, work on raw arrays too
int raw[4]{};
std::sort(std::begin(raw), std::end(raw));
```

`advance` and `distance` are $O(1)$ on random access and $O(n)$ otherwise. Calling `distance` in a loop over a `std::list` is a quiet way to turn an $O(n)$ pass into $O(n^2)$.

1.3 const, reverse, and the base() trap

```
for (auto it = v.cbegin(); it != v.cend(); ++it) // const_iterator, cannot write
    std::cout << *it;

for (auto it = v.rbegin(); it != v.rend(); ++it) // walks 50,40,30,20,10
    std::cout << *it;
```

A reverse iterator stores the position *one past* the element it dereferences. So `&*rit == &(rit.base() - 1)`. This matters the moment you convert one to pass into `erase`:

```
// erase the last element equal to 30
auto rit = std::find(v.rbegin(), v.rend(), 30);
if (rit != v.rend())
    v.erase(std::next(rit).base()); // NOT rit.base()
```

1.4 Invalidation

This is the single most asked iterator question, and the answer differs per container. Note the split between *iterators* and *references/pointers*; they are not always invalidated together.

Container	Insert	Erase
vector, string	If capacity is exceeded, everything is invalidated. Otherwise iterators from the insert point onward. <code>end()</code> always.	Iterators at and after the erased element.
deque	All iterators invalidated. References survive if you inserted at either end.	At an end: only the erased one. In the middle: all iterators and references.
list, forward_list	Nothing invalidated.	Only the erased element.
set, map, multiset, multimap	Nothing invalidated.	Only the erased element.
unordered_*	Rehash invalidates all iterators; references and pointers survive. No rehash means nothing is invalidated.	Only the erased element.

Two consequences people trip over. First, `push_back` invalidates `end()` even when no reallocation happens, so a cached end sentinel is always wrong. Second, for deque the reference guarantee and the iterator guarantee genuinely differ: `push_front` keeps your `int&` alive but poisons your iterator.

1.5 Erasing while looping

```
// wrong: it is dangling the moment erase returns
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it % 2 == 0) v.erase(it); // UB

// right: erase hands back the next valid position
for (auto it = v.begin(); it != v.end(); )
    if (*it % 2 == 0) it = v.erase(it);
    else ++it;
```

For node-based containers before C++11 the idiom was `m.erase(it++)`, which works because the post-increment is sequenced before the call. Since C++11 `map::erase` also returns the next iterator, so use the same shape as above and stop thinking about it.

For a vector, do not hand-roll the loop at all. Erase-remove is one reallocation-free pass:

```
v.erase(std::remove_if(v.begin(), v.end(),
    [](int x){ return x % 2 == 0; }),
    v.end());

// C++20
std::erase_if(v, [](int x){ return x % 2 == 0; });
```

`remove_if` alone does not shrink the container. It shuffles the keepers to the front and returns the new logical end; the tail is left in a valid but unspecified state. Forgetting the second erase call is the classic bug.

1.6 Inserters and stream iterators

Output iterators let algorithms write into something that has no room yet. `std::copy` does not resize the destination, so this is how you avoid writing past the end:

```
std::vector<int> dst;
std::copy(src.begin(), src.end(), std::back_inserter(dst)); // push_back
std::copy(src.begin(), src.end(), std::inserter(s, s.end())); // set/map

std::copy(v.begin(), v.end(),
    std::ostream_iterator<int>(std::cout, " "));

std::vector<int> nums{std::istream_iterator<int>(std::cin),
    std::istream_iterator<int>()); // default = EOF
```

That last line, if you write it as a named variable, is the most-vexing-parse trap. `std::vector<int> nums(std::istream_iterator<int>(std::cin), std::istream_iterator<int>());` declares a function. Braces avoid it.

1.7 Writing one

A minimal forward iterator needs five type aliases (so `iterator_traits` can see them) plus `*`, `++` and `==`.

```
class Range {
    int lo_, hi_;
public:
    Range(int lo, int hi) : lo_(lo), hi_(hi) {}

    struct iterator {
        using iterator_category = std::forward_iterator_tag;
        using value_type        = int;
        using difference_type    = std::ptrdiff_t;
        using pointer            = const int*;
        using reference          = const int&;

        int v;
        reference operator*() const { return v; }
        iterator& operator++()      { ++v; return *this; }
        iterator operator++(int)    { auto t = *this; ++v; return t; }
        bool operator==(const iterator& o) const { return v == o.v; }
        bool operator!=(const iterator& o) const { return !(*this == o); }
    };

    iterator begin() const { return {lo_}; }
    iterator end()   const { return {hi_}; }
};

for (int x : Range(0, 5)) std::cout << x;    // 01234
```

Inheriting from the old `std::iterator` template was the shortcut for this; it is deprecated since C++17, so write the aliases out. In C++20 the aliases are still what the `std::forward_iterator` concept checks against, and `operator!=` can be dropped because the compiler synthesises it.

1.8 Things that bite

- Comparing iterators from two different containers is undefined, not false.
- `*it` on `end()` is undefined even if the memory happens to be readable.
- A range-for holds `end()` internally, so mutating the container inside the loop body is the same bug as erasing during an explicit loop.
- `std::vector<bool>` is not a container of `bool`. `*it` gives a proxy object, not a `bool&`, so `auto& b = v[0]` does not do what it looks like.

2. std::stack

```
template<class T, class Container = std::deque<T>>
class stack;
```

Note what this is: a **container adapter**, not a container. It owns a container object (a protected member conventionally named `c`) and exposes a deliberately narrow interface over it. No iterators, no `clear()`, no `operator[]`, no range-for. That is the point: the restriction is the feature.

2.1 Interface

Call	Does	Cost	Returns
<code>push(x)</code>	copy or move <code>x</code> onto the top	$O(1)$ amortised	void
<code>emplace(args...)</code>	construct in place on top	$O(1)$ amortised	reference (C++17), void before
<code>pop()</code>	remove the top element	$O(1)$	void
<code>top()</code>	the top element	$O(1)$	<code>T&</code> / <code>const T&</code>
<code>empty()</code>	<code>size() == 0</code>	$O(1)$	bool
<code>size()</code>	element count	$O(1)$	<code>size_type</code>
<code>swap(other)</code>	exchange contents	$O(1)$	void

```
std::stack<int> s;
s.push(1);
s.push(2);
std::cout << s.top();    // 2
s.pop();                 // returns nothing
std::cout << s.size();   // 1

if (!s.empty()) s.pop();
s.pop();                 // UB on an empty stack, no exception, no check
```

2.2 Why pop() does not return the element

This is asked constantly and it is an exception-safety argument, not a style choice.

A hypothetical `T pop()` would have to remove the top element and then return it by value. Returning by value copies. If that copy constructor throws, the element has already been removed from the stack and the copy never reached the caller. The value is gone and there is no way to recover it. Splitting the operation into `top()` (which can throw harmlessly, nothing has changed yet) and `pop()` (which cannot throw) means every step is either safe to retry or safe to fail.

Secondary point, worth adding if they want more: returning by value costs a copy even for callers who only wanted to discard the element. The split interface makes you pay only for what you use.

```
int x = std::move(s.top()); // take it
s.pop();                  // then drop it
```

Do not write `auto& r = s.top(); s.pop(); use(r);`. `r` is dangling after the pop.

2.3 Choosing the underlying container

Anything with `back()`, `push_back()`, `pop_back()`, `empty()` and `size()` qualifies. So all three of the sequence containers work.

```

std::stack<int> a; // deque, the default
std::stack<int, std::vector<int>> b; // contiguous, fewer allocations
std::stack<int, std::list<int>> c; // stable addresses, no reallocation

std::vector<int> seed{1, 2, 3};
std::stack<int, std::vector<int>> d(seed); // 3 is on top

```

Backing	Trade-off
deque (default)	No reallocation of existing elements, grows in blocks, slightly worse locality and an extra indirection per access.
vector	Best cache behaviour and the usual choice when you know the depth. Growth reallocates and copies. Reserve first if you can.
list	Every element is a separate allocation. Only worth it when element addresses must stay put forever.

2.4 Reaching the container underneath

The member is protected, not private, which is a deliberate hook. Derive if you need to iterate for debugging:

```

template<class T, class C>
const C& innards(const std::stack<T, C>& s) {
    struct Peek : std::stack<T, C> {
        static const C& get(const std::stack<T, C>& st) {
            return st.*&Peek::c;
        }
    };
    return Peek::get(s);
}

```

Fine for a debug print. If production code needs this, the stack was the wrong type to pick.

2.5 Things that bite

- `top()` or `pop()` on an empty stack is undefined behaviour. There is no bounds check and no `at()` equivalent. Guard with `empty()`.
- No `clear()`. Use `s = {};` or swap with a fresh empty stack.
- `==`, `<` and friends are defined and forward to the underlying container, so two stacks compare lexicographically from the bottom up. C++20 gives it `<=>`.
- Two stacks with different container parameters are unrelated types. They will not compare or assign.

3. std::queue

```
template<class T, class Container = std::deque<T>>
class queue;
```

Same adapter idea, FIFO instead of LIFO. Push at the back, pop from the front, look at either end.

Call	Does	Cost	Returns
push(x)	add at the back	O(1) amortised	void
emplace(args...)	construct in place at the back	O(1) amortised	reference (C++17)
pop()	remove from the front	O(1)	void
front()	oldest element	O(1)	T& / const T&
back()	newest element	O(1)	T& / const T&
empty(), size()	as expected	O(1)	bool, size_type

```
std::queue<std::string> q;
q.push("a");
q.push("b");
std::cout << q.front(); // a
std::cout << q.back();  // b
q.pop();                // "a" is gone

// the standard drain loop, e.g. BFS
while (!q.empty()) {
    auto node = std::move(q.front()); // copy or move out first
    q.pop();
    process(node);
}
```

Take the element out by value before popping. Holding a reference across the pop() is the same dangling bug as with stack, and it is easy to hide behind auto&.

3.1 Why vector is not allowed here

A queue needs front(), back(), push_back() and pop_front(). std::vector has no pop_front, because removing from the front of a vector is O(n) and the standard will not offer an operation that quietly costs that much. So:

```
std::queue<int> a; // deque
std::queue<int, std::list<int>> b; // legal
std::queue<int, std::vector<int>> c; // ERROR: no pop_front
```

deque is the right default here for a real reason: it can grow at both ends without moving what is already stored, which is exactly the access pattern a queue has.

3.2 Things that bite

- No iterators, so no range-for and no std::find over a queue. If you need to search it, you needed a deque.
- front() on an empty queue is undefined, same as stack.
- Copying a queue copies the whole underlying container. In a BFS that pushes state objects, pass by reference and move elements in.

3.3 std::priority_queue

```
template<class T,
        class Container = std::vector<T>,
        class Compare = std::less<typename Container::value_type>>
class priority_queue;
```

Lives in the same header, and it is a binary heap over a vector rather than a queue in the FIFO sense. The default with `std::less` is a **max**-heap, which surprises people every single time.

Call	Cost	Note
<code>push / emplace</code>	$O(\log n)$	sift up
<code>pop</code>	$O(\log n)$	sift down, returns void
<code>top</code>	$O(1)$	const reference only, you cannot mutate the element
construct from a range	$O(n)$	heapify, cheaper than n pushes

```
std::priority_queue<int> maxh;                                // largest on top

std::priority_queue<int, std::vector<int>,
                    std::greater<int>> minh;                // smallest on top

// build from a range: one heapify pass, not n pushes
std::vector<int> data{5, 1, 9, 3};
std::priority_queue<int> h(data.begin(), data.end());

// custom order: "return true if a should come out AFTER b"
struct ByCost {
    bool operator()(const Job& a, const Job& b) const {
        return a.cost > b.cost;        // cheapest job on top
    }
};
std::priority_queue<Job, std::vector<Job>, ByCost> jobs;
```

The comparator reads backwards relative to `std::sort`. For `sort`, `cmp(a, b)` true means `a` goes first. For a `priority_queue` it means `a` has *lower* priority and comes out later. If your heap is inverted, this is why.

`top()` returns `const` because mutating an element in place would break the heap invariant with no way for the container to notice. To change a priority: `pop`, `modify`, `push`. Or push a new entry and skip stale ones on `pop`, which is the usual Dijkstra trick.

3.4 Picking between the three

Need	Use
LIFO, nothing else	stack
FIFO, nothing else	queue
Always want the extreme element	priority_queue
FIFO but you also need to scan or index the contents	deque directly
Fixed-capacity ring buffer	Neither. Roll it over a vector or an array.

4. Twenty questions

Short answers. Say the first sentence, then expand only if they wait.

Is `std::stack` a container?

No, it is a container adapter. It wraps a real container and exposes a restricted interface. That is why it has no iterators and no `clear()`.

What is the default underlying container for stack and queue?

`std::deque` for both. `priority_queue` defaults to `std::vector`.

Why does `pop()` return `void`?

Exception safety. Removing the element and returning it by value would be one operation with two failure points; if the return copy throws, the element is already gone and unrecoverable. Splitting into `top()` and `pop()` makes each half safe on its own.

Can I use vector for a stack? For a queue?

For a stack, yes. For a queue, no, because vector has no `pop_front` and removing from the front would be $O(n)$.

How do you clear a stack?

There is no `clear()`. Assign an empty one, `s = {}`, or swap with a temporary.

What happens if you call `top()` on an empty stack?

Undefined behaviour. There is no check and no exception. Test `empty()` first.

Name the iterator categories.

Input, output, forward, bidirectional, random access, and contiguous since C++20. Each one refines the previous.

Which category does `std::list` give you, and what does that stop you doing?

Bidirectional. It rules out `std::sort`, `std::nth_element`, binary search with useful complexity, and anything else needing $it + n$. `list` has its own `sort()` member for this reason.

Give a random-access container that is not contiguous.

`std::deque`. Constant-time indexing, but the elements live in separate blocks, so you cannot pass `&v[0]` to a C API expecting an array.

What invalidates vector iterators?

Any reallocation invalidates everything. Without reallocation, `insert` or `erase` invalidates iterators from that position onward. `push_back` always invalidates `end()`.

Same question for map and list.

Only the erased element is invalidated. Insertion invalidates nothing. That stability is often the reason to pick them.

And `unordered_map`?

A rehash invalidates all iterators but leaves references and pointers valid, because the nodes themselves do not move. Erase only invalidates what was erased.

Write a loop that erases every even number from a vector.

erase-remove: `v.erase(std::remove_if(v.begin(), v.end(), pred), v.end())`. Or `std::erase_if(v, pred)` in C++20. Erasing one at a time inside a manual loop is $O(n^2)$ and easy to get wrong.

Why does `remove_if` not actually remove anything?

Algorithms work on iterators and have no access to the container, so they cannot resize it. `remove_if` compacts the keepers to the front and returns the new logical end; you call `erase` to shrink.

What is wrong with: `for (auto it = v.begin(); it != v.end(); ++it) if (*it == x) v.erase(it);`

After `erase`, it is invalid, and both `++it` and the comparison against `end()` are undefined. Use `it = v.erase(it)` and only advance on the non-erasing branch.

What does `rit.base()` give you?

The forward iterator one position past the element `rit` refers to. To erase what `rit` points at, pass `std::next(rit).base()`.

What do you need to write your own iterator?

Five typedefs so `iterator_traits` works (`iterator_category`, `value_type`, `difference_type`, `pointer`, `reference`), plus `operator*`, `operator++` and equality. Deriving from `std::iterator` is deprecated in C++17, so spell them out.

Default `priority_queue`: max-heap or min-heap?

Max-heap. The default comparator is `std::less`, and the largest element sits on top. For a min-heap pass `std::greater` as the third template argument, which means also naming the container as the second.

Why is `priority_queue::top()` const?

Editing an element in place could violate the heap ordering with no way for the container to detect it. Pop, modify, push. Or push a duplicate with the new priority and discard stale entries when they surface.

Building a heap: `n` pushes or construct from a range?

Range construction, $O(n)$ heapify against $O(n \log n)$ for repeated pushes. Same reason `std::make_heap` exists.

Recall card

stack and queue: adapters over deque, no iterators, pop returns void, UB when empty. queue rejects vector (no `pop_front`). `priority_queue` is a max-heap over a vector, `top()` is const, range construction is $O(n)$. Iterator categories: input, output, forward, bidirectional, random access, contiguous. deque is random access but not contiguous. Vector reallocation kills every iterator; node containers only invalidate what you erased; unordered rehash kills iterators but spares references. Erase in a loop with `it = c.erase(it)`, erase a vector predicate with `erase-remove`.